Server Client Service Portal

Software Architecture and Technical Design Document

Project Name	Server Client Service Portal
Author	Manoa Robel
Post	Intern
Version	2.0
Date	August 2026
Document Type	Software Architecture and Technical Design Document

Contents

List of figures

List of tables

Summary

Version 2.0 of the Server Client Service Portal formalizes the backend around repository interfaces and replaceable persistence adapters. The REST API supports authenticated workflows for clients, engineers, and administrators, including request creation, assignment, status transitions, and public or internal comments. The active runtime remains backed by in-memory repositories, while PostgreSQL services and integration tests provide the migration target. This document describes the current architecture, security model, data design, API behavior, deployment configuration, test strategy, known limitations, and the remaining work required to complete the persistence migration.

1. Introduction

1.1 Purpose

This document provides a comprehensive architectural and technical description of the Server Client Service Portal. It serves as a reference for technical leads, and internship supervisors responsible for evaluating, maintaining platform.

1.2 Scope

The document covers the application architecture, technology stack, security model, data-management strategy, API design, test approach, runtime configuration, build process, and current implementation state. It is limited to capabilities implemented or explicitly described in the source project documentation.

1.3 Objectives

- Provide a centralized platform for managing service requests.
- Enable secure authentication and role-based authorization.
- Support role-specific workflows for clients, engineers, and administrators.
- Maintain traceability of request assignments, status changes, and comments.
- Support migration from volatile in-memory storage to PostgreSQL while preserving application behavior.

2. Project Overview

The Server Client Service Portal is a REST API for managing service requests across structured workflows involving clients, engineers, and administrators. It provides user management, request creation and tracking, engineer assignment, request status history, and public or internal comments. JSON Web Tokens provide stateless authentication, and Role-Based Access Control policies enforce endpoint permissions.

The application currently runs against an in-memory data store. A complete PostgreSQL implementation exists under the service and database layers and is covered by database integration tests, but it is not yet connected to the running HTTP application.

3. Functional Overview

3.1 User Management

The platform supports three roles. Clients create and monitor their service requests. Engineers process requests and participate in operational workflows. Administrators manage users, requests, assignments, and workflow administration.

3.2 Service Request Management

The application supports creation, retrieval, content updates, status transitions, assignment, deletion, and lifecycle tracking of service requests. Access to each operation is controlled by the authenticated user's role and relationship to the target request.

3.3 Comment Management

Comments support collaboration among stakeholders. Clients may add comments that are automatically public and may view public comments on their own requests. Engineers and administrators may create and view both public and internal comments.

3.4 Assignment Management

Administrators may assign requests to users whose role is engineer. Assignment data is represented independently from the request record to preserve relationship history and enforce domain constraints.

4. System Architecture

4.1 Architecture Style

The application follows a layered architecture that separates HTTP delivery, business logic, and persistence responsibilities. The primary execution path is Controllers (HTTP/Express) → Services (business logic) → Database.

4.2 Layer Responsibilities

Delivery layer. Controllers, middleware, and schemas receive HTTP requests, authenticate and authorize callers, validate payloads, invoke services, and construct responses.

Service layer. The service layer encapsulates business rules for users, requests, assignments, comments and workflow transitions. Business services depend on repository contracts defined in src/core/ports.ts rather than concrete persistence structures. In-memory adapters in src/core/in-memory.repositories.ts implement these contracts over database.ts, aligning the active implementation with the dependency injection model used by PostgreSQL services.

Data layer. The data layer consists of in-memory repository adapters and PostgreSQL persistence services. Both implementations represent the same domain model and support the ongoing persistence migration strategy.

4.3 Data Flow

1. A client submits an HTTP request.
2. Middleware authenticates and authorizes the caller.
3. Zod schemas validate request data.
4. The controller invokes the relevant business service.
5. The service applies domain rules and accesses the active persistence implementation.
6. The controller returns the resulting HTTP response.

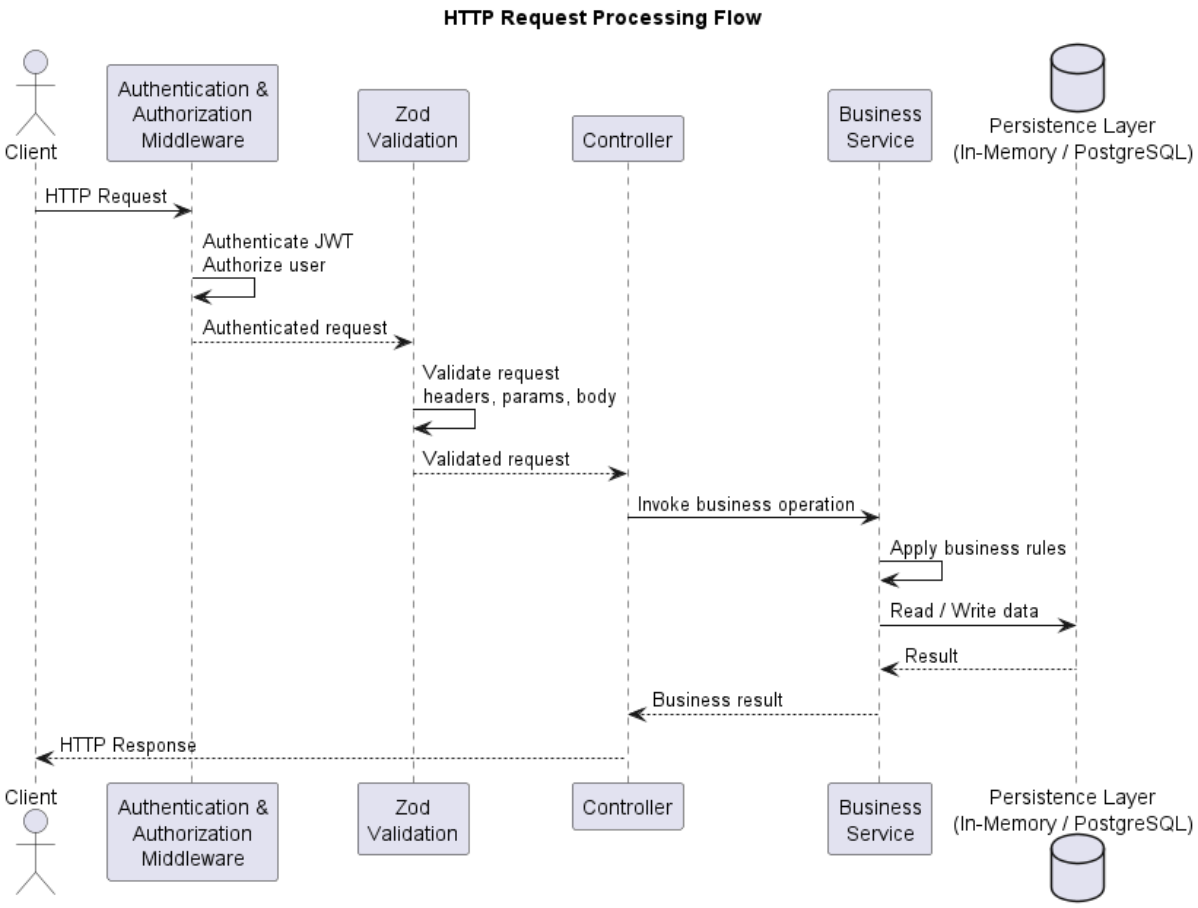

Figure 1: HTTP Request Processing Flow

4.4 Component Interactions

Controllers remain independent of server startup, enabling tests to import the Express application without binding a network port. Configuration modules validate runtime settings and configure API documentation. Database modules manage pooling, schema initialization, and persistence infrastructure.

5. Technology Stack

Technology	Version	Purpose	Reason for Selection
Node.js	22.22.3	Application runtime	Provides an efficient runtime for HTTP services and includes the native test runner used by the project

TypeScript	Strict compiler mode	Implementation language	Provides static typing and compile-time validation
Express	5.2.1	REST API framework	Provides routing and middleware composition with a focused HTTP abstraction.
PostgreSQL	18.4	Relational persistence	Provides native UUIDs, ENUMs, constraints, indexes, and transactional tests.
pg	8.22.0	Database connectivity	Provides native PostgreSQL access and TCP connection pooling.
Zod	4.4.3	Schema-first validation	Validates environment variables and HTTP request bodies with shared schemas.
JWT	9.0.3	Authentication	Enables stateless identity propagation across protected endpoints.
bcrypt	6.0.0	Password hashing	Protects stored credentials through one-way hashing.
Helmet	8.3.0	HTTP header security	Applies protective HTTP response headers.
CORS	2.8.6	Cross-origin control	Restricts browser access to configured origins.
node:test	Native Node.js	Automated testing	Provides a lightweight built-in test runner.

tsx	4.23.1	Development execution	Runs TypeScript directly and supports watch-based reload.
@asteasolutions/zod-to-openapi	9.1.0	API documentation	Generates OpenAPI definitions from Zod schemas.

Tableau 1: Technology Stack

6. Project Structure

6.1 Application Entry Points

src/app.ts defines the Express application, including routes and middleware, without starting an HTTP server. This separation allows automated tests to import the application without opening a port

src/server.ts starts the HTTP server and manages graceful shutdown

6.2 Configuration

src/config centralizes runtime configuration. env.config.ts validates environment variables with Zod, while docs.config.ts configures OpenAPI and Swagger documentation

6.3 Active In-Memory Core

src/core contains the active in-memory implementation. database.ts stores seed and runtime data. The user, request, and comment services implement domain operations. id.util.ts generates UUIDs, and date.util.ts formats API dates

6.4 Delivery Layer

src/delivery contains authentication, authorization, and validation middleware; Zod request and response schemas; and HTTP controllers. This directory defines the external API boundary

6.5 PostgreSQL Services

src/services contains PostgreSQL-backed service implementations that mirror current in-memory behavior. This layer is complete but not yet connected to the running application

6.6 Database Infrastructure

src/db contains postgres.ts for connection pooling, schema.sql for relational schema definition, and init-db.ts for database creation and schema initialization.

6.7 Automated Tests

tests contains auth.test.ts and requests-matrix.test.ts for HTTP behavior against the in-memory application ; user-service.test.ts and request.service.test.ts for PostgreSQL integration testing with transaction isolation.

7. Authentication & Authorization

7.1 JWT

The platform uses JSON Web Tokens for stateless authentication. Tokens are signed with the configured JWT_SECRET. Protected requests are processed by authentication middleware that validates the token and exposes the authenticated user through req.user.

7.2 RBAC

Role-Based Access Control recognizes client, engineer, and admin roles. Permissions are evaluated for every protected endpoint. The application never trusts role information supplied in a request body; identity and role are derived exclusively from the validated JWT.

7.3 Authentication Flow

1. The user submits credentials to POST /api/auth/login.
2. The application verifies the submitted password against the stored bcrypt hash.
3. The application signs and returns a JWT access token.
4. Subsequent requests provide the token for middleware validation.
5. Validated identity information is attached to the request context.

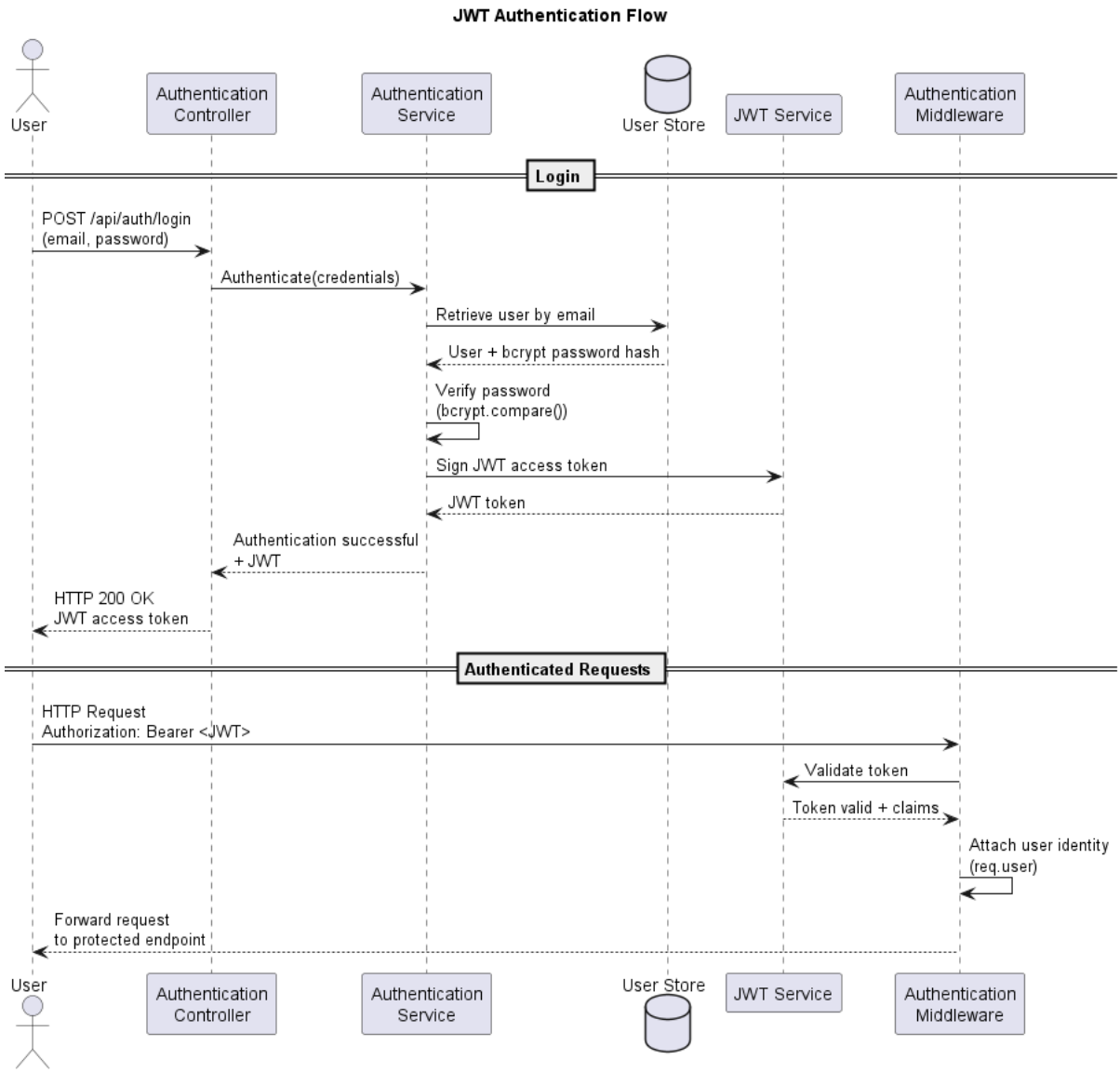

Figure 2: Authentication Flow

7.4 Authorization Flow

After authentication, authorization middleware evaluates the caller's role and, where applicable, ownership or assignment relationship. Clients are constrained to their own requests, engineers receive operational access according to request and endpoint rules, and administrators receive management permissions.

7.5 Access Rules

- **GET /api/requests:** clients view their own requests; engineers view assigned requests; administrators view all requests.
- **POST /api/requests:** permitted only for clients.
- **GET /api/requests/:id:** clients access their own requests; engineers and administrators may access any request.
- **PUT /api/requests/:id:** clients may edit title, description, and priority on their own requests; engineers and administrators may edit any request.
- **PATCH /api/requests/:id:** clients are denied; engineers may transition open requests to resolved; administrators may perform any valid transition, including transitions to closed.
- **DELETE /api/requests/:id:** permitted only for administrators.
- **POST /api/requests/:id/assignments:** permitted only for administrators, and the assignee must be an engineer.
- **GET /api/requests/:id/comments:** clients may view public comments on their own requests; engineers and administrators may view all comments.
- **POST /api/requests/:id/comments:** clients may add public comments; engineers and administrators may add public or internal comments.
- **GET /api/users:** permitted only for administrators.
- **GET /api/users/:id:** permitted for engineers and administrators.
- **POST, PATCH, and DELETE /api/users:** user creation, modification, and deletion are restricted to administrators. Self-service profile editing is intentionally disabled.

8. Data Layer

8.1 Current In-Memory Implementation

The running application uses the in-memory store in src/core/database.ts. Controllers under src/delivery call the corresponding in-memory services. All stored data is reset when the process restarts.

8.2 PostgreSQL Implementation

A complete PostgreSQL-backed implementation exists in src/services/user.service.ts, src/services/request.service.ts, and src/db. Its behavior is covered by integration tests against a live database.

8.3 Migration Strategy

The database-backed services mirror the logic implemented under src/core. Migration requires connecting the delivery layer to the PostgreSQL services while preserving existing API contracts, RBAC behavior, domain rules, and test coverage.

8.4 Coexistence Rationale

Both implementations coexist because the PostgreSQL layer has been developed and tested independently but has not yet replaced the active in-memory path. This parallel state reduces migration risk by allowing database behavior to be verified before runtime integration.

9. Database Design

9.1 Tables

The relational model includes users, requests, assignments, comments, and request status history. Assignments and status history are represented by dedicated tables rather than columns on the request table. The users table includes an is_active column to align with the domain model.

9.2 Relationships

Users act as clients, engineers, or administrators. Requests reference their client owners. Assignments associate requests with engineers. Comments reference requests and their authors. Status-history records reference requests and preserve workflow changes.

9.3 UUID Strategy

UUIDs identify records throughout the application. The in-memory implementation uses crypto.randomUUID(), while PostgreSQL uses gen_random_uuid(), providing consistent identifier semantics across both data layers.

9.4 ENUM Usage

PostgreSQL native ENUM types define user roles, request statuses, and priorities. The request-status ENUM includes pending_client, although current workflows do not support that state comprehensively.

9.5 Indexes

Five B-tree indexes support expected lookup and relationship-navigation patterns: users(email), requests(client_id), assignments(engineer_id), comments(request_id), and request_status_history(request_id).

9.6 Referential Integrity

Foreign-key constraints preserve valid relationships and use either cascading deletion or nullification according to the affected table's lifecycle requirements.

10. API Design

10.1 REST Principles

The API uses resource-oriented routes and standard HTTP methods to represent operations on authentication, users, requests, assignments, and comments. Route behavior is constrained by role and resource relationship.

10.2 Endpoint Organization

Authentication endpoints issue access tokens. Request endpoints implement request lifecycle operations. Nested assignment and comment endpoints represent resources associated with a request. User endpoints provide administrative and authorized lookup operations.

10.3 Validation

Zod provides schema-first validation for environment variables and HTTP request bodies. Delivery schemas define valid inputs and responses before business services are invoked.

10.4 Error Handling

Authentication, authorization, and validation middleware reject invalid or unauthorized requests before controller business logic executes. This centralization produces consistent failure behavior across endpoints.

10.5 OpenAPI Documentation

The API is documented with @asteasolutions/zod-to-openapi. Swagger UI is available at GET /api-docs, and the raw OpenAPI JSON specification is available at GET /api-docs.json.

10. Environment Configuration

Variable	Purpose	Example or Constraint
PORT	HTTP listening port	3001
NODE_ENV	Runtime environment	development, production, or test
URL_SITE	Origin used to construct production CORS policy	localhost:3001/
JWT_SECRET	Secret used to sign JWT access tokens	Minimum recommended length: 10 characters
PASSWORD_HASH	bcrypt hash used by seeded demonstration accounts	Required; no default value
DB_HOST	PostgreSQL host	Default exists in code
DB_PORT	PostgreSQL port	5432
DB_USER	PostgreSQL username	postgres
DB_PASSWORD	PostgreSQL password	Environment-specific
DB_NAME	PostgreSQL database name	service_portal

Tableau 2: Environment Configuration Table

The server does not start without PASSWORD_HASH because seeded demonstration accounts in the active in-memory database authenticate against this hash. Database variables already have code defaults and are required only by the PostgreSQL layer, which is not yet connected to the running application.

11. Build & Execution

11.1 Installation

Clone the repository, enter the project directory, and install dependencies:

- git clone --single-branch –branch backend https://github.com/Robel-Manoa/client-service-portal.git
- cd client-service-portal
- npm install

11.2 Available Scripts

- npm run dev : starts the development server with automatic reloading through tsx watch.
- npm run build : compiles TypeScript into dist/.
- npm start : starts the compiled application with node dist/server.js.
- npm test : runs all tests/*.test.ts files with the native Node.js test runner.
- npm run db:init : creates the PostgreSQL database if required and applies src/db/schema.sql.

11.3 Database Initialization

The database initialization script is idempotent and self-contained. It prepares the service_portal database and applies the current DDL schema without requiring manual schema execution.

12. Testing Strategy

12.1 Test Philosophy

The test strategy verifies the active application behavior and the future PostgreSQL service layer independently. This separation protects existing HTTP behavior while validating the migration target against a real database.

12.2 Unit and Behavioral Tests

auth.test.ts and requests-matrix.test.ts issue real HTTP calls against the Express application without accessing PostgreSQL. They cover login, role-based permissions, request access, status transitions, comments, and related workflow behavior.

12.3 Integration Tests

user-service.test.ts and request.service.test.ts exercise PostgreSQL-backed services against a live database and verify real SQL behavior.

12.4 Transaction Isolation

Each database integration test opens a dedicated connection, starts a transaction with BEGIN, executes test logic, and completes with ROLLBACK, including when the test fails. This approach prevents residual data from being persisted after the suite runs.

13. Security Considerations

13.1 Password Hashing

bcrypt protects passwords through one-way hashing. Demonstration accounts authenticate against the hash supplied through PASSWORD_HASH.

13.2 JWT

JWT provides stateless caller authentication. Tokens are signed using JWT_SECRET, and identity claims are validated before protected operations execute.

13.3 Zod Validation

Zod rejects malformed configuration and request payloads at the system boundary, limiting invalid data propagation into business logic.

13.4 Helmet

Helmet applies protective HTTP headers to reduce exposure to common browser-based attack vectors.

13.5 CORS

CORS configuration restricts browser-origin access according to the configured site URL.

13.6 Principle of Least Privilege

Role-specific permissions grant only the capabilities required for each role. Clients operate on their own requests, engineers receive constrained operational access, and administrative operations remain restricted to administrators. Role information is taken from the authenticated token rather than request payloads.

14. Current Limitations

1. The running server still depends on the in-memory implementation rather than PostgreSQL persistence.
2. In-memory data is lost whenever the application process restarts.
3. The PostgreSQL implementation exists but is not yet connected to the delivery layer.
4. The request-status ENUM includes pending_client, but current workflows do not comprehensively support that state.
5. Engineers currently support only the open → resolved status transition.
6. Self-service profile editing is intentionally disabled.

15. Future Improvements

15.1 PostgreSQL Integration

The next migration step will connect the existing PostgreSQL services to the running application and replace the active in-memory persistence path while preserving established API behavior.

15.2 Workflow Expansion

Future workflow changes will extend operational support for the pending_client request status beyond its current schema-level definition.

Conclusion

The Server Client Service Portal is a structured service-request management system built around layered responsibilities, schema-first validation, role-based security, and documented REST interfaces. The current in-memory application provides functional HTTP workflows, while the parallel PostgreSQL implementation establishes the persistence architecture targeted for integration. Clear separation of application startup, delivery concerns, business services, database infrastructure, and automated tests provides a maintainable foundation for completing the migration and extending the documented request lifecycle.

Appendix

Version 2.0 Architectural Updates

Version 2.0 introduces a repository-based ports-and-adapters architecture within src/core. Services now depend on repository interfaces defined in src/core/ports.ts and default to adapters implemented in src/core/in-memory.repositories.ts

The active execution path is now Controller -> Service -> Repository Interface -> Repository Adapter -> Persistence Store. This change improves separation of concerns, testability, maintainability and alignment with the PostgreSQL implementation.

A third testing layer has been added. tests/core/user.service.test.ts performs isolated unit testing of UserService using fake repository implementations without Express, HTTP infrastructure or PostgreSQL

During development, the platform was stabilized through fixes to environment validation, authentication middleware, controller defects, UUID adoption, schema corrections, RBAC implementation, OpenAPI coverage, repository abstraction and automated testing improvements